

CloudFront Origin Group – Geographical Failover with Load Balancers

To Begin with the Lab

Summary of the Lab

In this lab, Amazon CloudFront Origin Groups were configured to provide automatic geographical failover between Application Load Balancers deployed in Mumbai and N. Virginia. The Mumbai load balancer acted as the primary origin while the N. Virginia load balancer served as the backup origin. A CloudFront distribution was created and configured with both origins inside an origin group, allowing CloudFront to switch traffic automatically when the primary origin became unavailable. The failover process was tested by disabling HTTP access on the Mumbai load balancer and verifying that CloudFront redirected requests to the US load balancer. Finally, the primary origin was restored and CloudFront resumed serving traffic from Mumbai.

Understanding Geographical Failover

Geographical failover is a disaster recovery strategy where workloads are duplicated across multiple AWS Regions. If one region experiences an outage, traffic is automatically redirected to infrastructure running in another region. This ensures users continue accessing the application even during regional failures.

Example:

An e-commerce application serves customers from Mumbai under normal conditions. During a regional outage affecting ap-south-1, CloudFront routes requests to a backup environment hosted in us-east-1, allowing customers to continue browsing and purchasing products.

Understanding Failover Testing

Failover testing validates that CloudFront correctly switches traffic to the backup origin when the primary origin becomes unavailable. This ensures disaster recovery mechanisms function as expected before production deployment.

Example:

Removing inbound HTTP access from the Mumbai load balancer prevents it from receiving requests. CloudFront detects the failure and starts serving traffic from the Virginia load balancer.

Lab Steps

1. Prepare Regional Web Servers

- Sign in to the AWS Management Console.
- Launch two EC2 web servers in **Mumbai (ap-south-1)**.

- EC2 > Instances > Launch an instance

Name and tags

Name

web-server

Add additional tags

Application and OS Images (Amazon Machine Image)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

RecentsMy AMIsQuick Start

Amazon Linux

macOS

Ubuntu

Windows

Red Hat

SUSE Linux

Debian

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Summary

Number of instances1

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet. Data transfer charges are not included as part of the free tier

Cancel

Launch instance

Preview code

- ```
C:\Users\Shashank>cd downloads
```
- ```
C:\Users\Shashank\Downloads>ssh -i June-2026.pem ec2-user@65.2.73.108
```
- ```
#_
Amazon Linux 2023
#.#.#.#.#|
#.#.#.|
#/#/ https://aws.amazon.com/linux/amazon-linux-2023
V#/' -->
 _-'
 /m/'
[mec2-user@ip-172-31-5-100 ~]$
```

- Now login as root user by typing this command: **sudo -i**.
- Use **yum install httpd** command to configure and place webpage in Apache on the EC2 instance.

```
root@ip-172-31-6-87:~
[ec2-user@ip-172-31-6-87 ~]$ sudo -i
[root@ip-172-31-6-87 ~]# yum install httpd
Amazon Linux 2023 Kernel Livepatch Repository 392 kB/s | 47 kB 00:00
Last metadata expiration check: 0:00:01 ago on Fri Jun 12 09:07:49 2026.
Dependencies resolved.
=====
```

| Package                       | Architecture | Version                | Repository  | Size  |
|-------------------------------|--------------|------------------------|-------------|-------|
| Installing:                   |              |                        |             |       |
| httpd                         | x86_64       | 2.4.67-1.amzn2023.0.1  | amazonlinux | 46 k  |
| Installing dependencies:      |              |                        |             |       |
| apr                           | x86_64       | 1.7.5-1.amzn2023.0.4   | amazonlinux | 129 k |
| apr-util                      | x86_64       | 1.6.3-1.amzn2023.0.2   | amazonlinux | 97 k  |
| apr-util-ldb                  | x86_64       | 1.6.3-1.amzn2023.0.2   | amazonlinux | 13 k  |
| generic-logos-httpd           | noarch       | 18.0.0-12.amzn2023.0.3 | amazonlinux | 19 k  |
| httpd-core                    | x86_64       | 2.4.67-1.amzn2023.0.1  | amazonlinux | 1.4 M |
| httpd-filesystem              | noarch       | 2.4.67-1.amzn2023.0.1  | amazonlinux | 12 k  |
| httpd-tools                   | x86_64       | 2.4.67-1.amzn2023.0.1  | amazonlinux | 80 k  |
| libbrotli                     | x86_64       | 1.0.9-4.amzn2023.0.2   | amazonlinux | 315 k |
| mailcap                       | noarch       | 2.1.49-3.amzn2023.0.3  | amazonlinux | 33 k  |
| Installing weak dependencies: |              |                        |             |       |
| apr-util-openssl              | x86_64       | 1.6.3-1.amzn2023.0.2   | amazonlinux | 15 k  |
| mod_http2                     | x86_64       | 2.0.39-1.amzn2023.0.1  | amazonlinux | 167 k |
| mod_lua                       | x86_64       | 2.4.67-1.amzn2023.0.1  | amazonlinux | 59 k  |

```
Transaction Summary
=====
```

| Install 13 Packages        |  |
|----------------------------|--|
| Total download size: 2.4 M |  |

- Replace the default web page with the provided index.html content with the help of vi editor.

```
Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-ldb-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
httpd-core-2.4.67-1.amzn2023.0.1.x86_64
httpd-filesystem-2.4.67-1.amzn2023.0.1.noarch
httpd-tools-2.4.67-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mailcap-2.1.49-3.amzn2023.0.3.noarch
mod_lua-2.4.67-1.amzn2023.0.1.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
httpd-2.4.67-1.amzn2023.0.1.x86_64
httpd-filesystem-2.4.67-1.amzn2023.0.1.noarch
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mod_http2-2.0.39-1.amzn2023.0.1.x86_64
Completed
[root@ip-172-31-5-100 ~]# cd /var/www/html/
[root@ip-172-31-5-100 html]# vi index.html
```

- Use the above commands to change content of the default index.html and paste your code like this or use the code below.

```
root@ip-172-31-5-100:/var/www x + v
<!DOCTYPE html>
<html>
<head>
 <title>CloudFront Origin Group Demo</title>
 <style>
 body {
 font-family: Arial, sans-serif;
 background-color: #232F3E;
 color: white;
 text-align: center;
 padding-top: 100px;
 }
 .container {
 width: 70%;
 margin: auto;
 }
 h1 {
 color: #FF9900;
 }
 .status {
 font-size: 24px;
 color: #00FF7F;
 }
 </style>
</head>
<body>
 <div class="container">
 <h1>CloudFront Origin Group Lab</h1>
 <h2>Primary Origin - Amazon EC2</h2>
 <p class="status">Website is being served from the EC2 Web Server.</p>
 <p>This page demonstrates CloudFront Origin Group failover.</p>
 <p>If this server becomes unavailable, CloudFront will automatically redirect traffic to the S3 backup website.</p>
 </div>
</body>
</html>
..
..
..
..
35,7 ALL
```

```
<!DOCTYPE html>
<html>
<head>
 <title>CloudFront Origin Group Demo</title>
 <style>
 body {
 font-family: Arial, sans-serif;
 background-color: #232F3E;
 color: white;
 text-align: center;
 padding-top: 100px;
 }
 .container {
 width: 70%;
 margin: auto;
 }
 h1 {
 color: #FF9900;
 }
 .status {
 font-size: 24px;
 color: #00FF7F;
```

```

 }
 </style>
</head>
<body>
 <div class="container">
 <h1>CloudFront Origin Group Lab</h1>
 <h2>Amazon EC2 </h2>
 <p class="status">Website is being served from the EC2 Web Server.</p>
 </div>
</body>
</html>

```

- Now enable and run the web server services using these commands

```

systemctl start httpd
systemctl enable httpd

```

```

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
httpd-core-2.4.67-1.amzn2023.0.1.x86_64
httpd-tools-2.4.67-1.amzn2023.0.1.x86_64
mailcap-2.1.49-3.amzn2023.0.3.noarch
mod_lua-2.4.67-1.amzn2023.0.1.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
httpd-2.4.67-1.amzn2023.0.1.x86_64
httpd-filesystem-2.4.67-1.amzn2023.0.1.noarch
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mod_http2-2.0.39-1.amzn2023.0.1.x86_64

Complete!
[root@ip-172-31-5-100 ~]# cd /var/www/html/
[root@ip-172-31-5-100 html]# vi index.html
[root@ip-172-31-5-100 html]# systemctl start httpd
[root@ip-172-31-5-100 html]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@ip-172-31-5-100 html]#

```

- Verify that the EC2 website opens correctly using the public IP.
- Verify all web servers are accessible.

## 2. Create Security Groups

- Click on **EC2** → **Security Groups** → **Create security group**.
- In both regions, create an **ALB-SG** security group that allows HTTP traffic from anywhere on port 80, so configure the name, description and choose default VPC.

**Create security group** Info

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

**Basic details**

**Security group name** Info  
ALB-SG  
Name cannot be edited after creation.

**Description** Info  
Secure ALB

**VPC** Info  
vpc-00fb319f9736cdf16 (VPC-India)

- Choose proper **Inbound** and **Outbound** rule as shown and click on **Create security group**.

**Inbound rules** Info

Type	Protocol	Port range	Source	Description - optional
HTTP	TCP	80	Anywh...	

0.0.0.0/0

**Outbound rules** Info

Type	Protocol	Port range	Destination	Description - optional
All traffic	All	All	Custom	

0.0.0.0/0

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Rules with destination of 0.0.0.0/0 or ::/0 allow your instances to send traffic to any IPv4 or IPv6 address. We recommend setting security group rules to be more restrictive and to only allow traffic to specific known IP addresses.

- In both regions, create a **Web-Server-SG** security group that allows HTTP traffic only from the ALB security group with **Inbound** and **Outbound Rules** as shown.
- Select **ALB-SG** as the **Source** in **Inbound Rule** while creating Web-Server-SG.
- Add an SSH inbound rule to **WebServer-SG** from 0.0.0.0/0 so the instances can be managed only if needed.

**Edit inbound rules** [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

Security group rule ID	Type <a href="#">Info</a>	Protocol <a href="#">Info</a>	Port range <a href="#">Info</a>	Source <a href="#">Info</a>	Description - optional <a href="#">Info</a>	
sgr-05357bc99cb1a257c	HTTP	TCP	80	Cus...  sg-086d15fec408		<a href="#">Delete</a>
-	SSH	TCP	22	Any...  0.0.0.0		<a href="#">Delete</a>

[Add rule](#)

⚠ Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

[Cancel](#) [Preview changes](#) [Save rules](#)

- Launch **Web-Server-1** using **Amazon Linux**, **t3.micro** and do the same for other servers.

**Name and tags** [Info](#)

Name  [Add additional tags](#)

**Application and OS Images (Amazon Machine Image)** [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Recents | My AMIs | **Quick Start**

Amazon Linux

macOS

Ubuntu

Windows

Red Hat

SUSE Linux

[Browse more AMIs](#)  
Including AMIs from AWS, Marketplace and the Community

**Amazon Machine Image (AMI)**

Amazon Linux 2023 kernel-6.1 AMI  
ami-0db56f446d44f2f09 (64-bit (x86), uefi-preferred) / ami-07c18f602ca3306b9 (64-bit (Arm), uefi)  
Virtualization: hvm ENA enabled: true Root device type: ebs [Free tier eligible](#)

**Summary**

Number of instances [Info](#)

**Software Image (AMI)**  
Amazon Linux 2023 AMI 2023.12....[read more](#)  
ami-0db56f446d44f2f09

**Virtual server type (instance type)**  
t3.micro

**Firewall (security group)**  
Web-Server-SG

**Storage (volumes)**  
1 volume(s) - 8 GiB

**Free tier:** In your first year of opening an AWS account, you get 750 hours per month of t2.micro

[Cancel](#) [Launch instance](#) [Preview code](#)

- Repeat the same two EC2 launches in the US region.
- Navigate to **Target Groups** and create **TG-India** with HTTP port 80 in **default VPC**.

**Target group name**  
Name must be unique per Region per AWS account.  
TG-India

**Protocol**  
Protocol for communication between the load balancer and targets.  
HTTP

**Port**  
Port number where targets receive traffic. Can be overridden for individual targets during registration.  
80

**IP address type**  
Only targets with the indicated IP address type can be registered to this target group.  
☒ IPv4  
Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.  
☐ IPv6  
Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). [Learn more](#)

**VPC**  
Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.  
vpc-Od1bb5239b4840bd3 (VPC-India)  
192.168.100.0/24

- Register both India EC2 instances as targets and click **Include as pending below** and click **Next**.

**Register targets - recommended**  
This is an optional step to create a target group. However, to ensure that your load balancer routes traffic to this target group you must register your targets.

**Available instances (2/2)**

Instance ID	Name	State	Security groups	Zone
i-00193c80bf57b50fe	Web-Server-2	Running	Web-Server-SG	ap-so
i-0b2dc2d16c581637f	Web-Server-1	Running	Web-Server-SG	ap-so

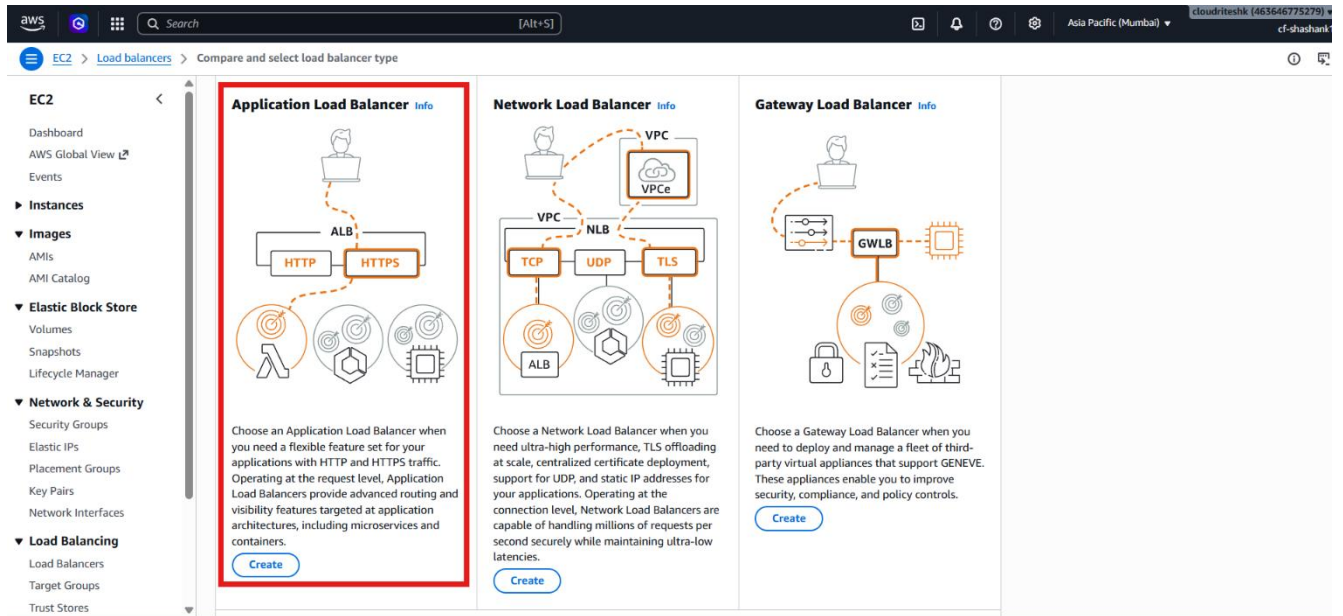
2 selected

**Ports for the selected instances**  
Ports for routing traffic to the selected instances.  
80  
1-65535 (separate multiple ports with comma)

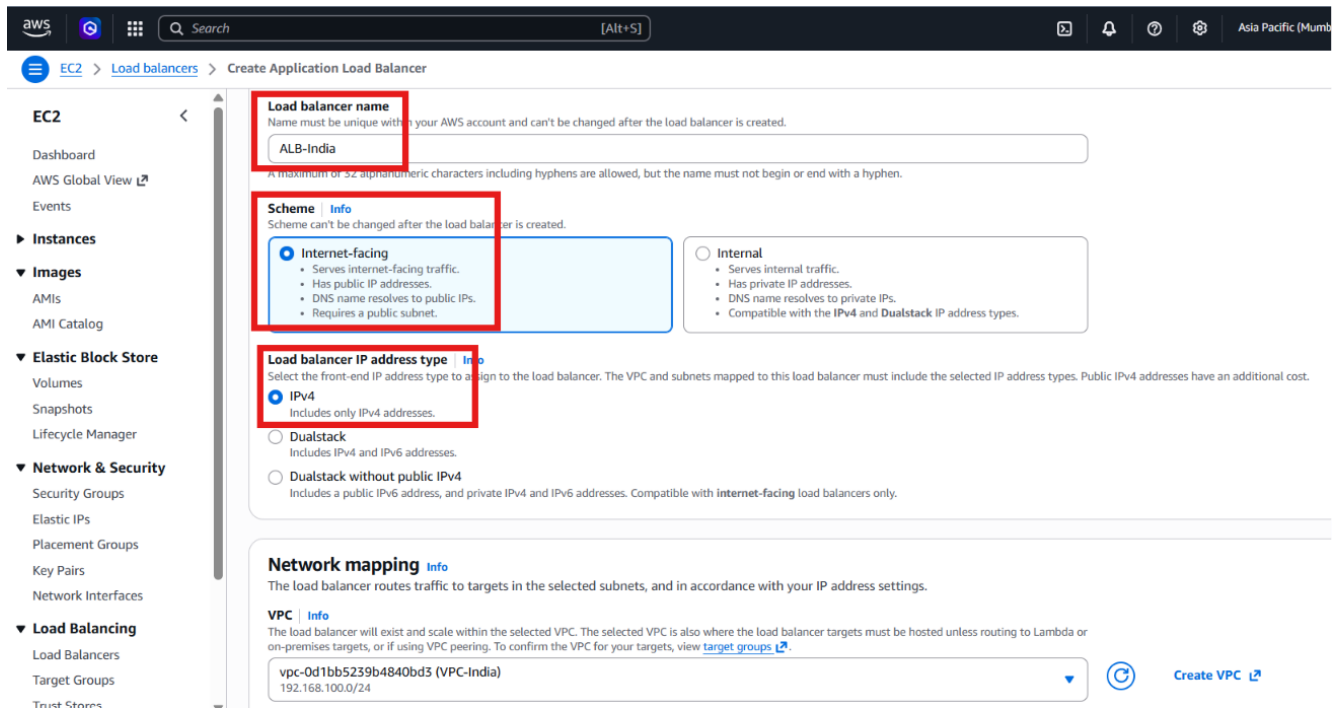
[Include as pending below](#)

- Wait for the target health to become healthy.
- Now go to **Load Balancer** and choose **Application Load Balancer**.





- Create an Application Load Balancer named **ALB-India** in the public subnets of **VPC-India** as shown below.



EC2 > Load balancers > Create Application Load Balancer

**Network mapping** Info

The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.

**VPC** Info

The load balancer will exist and scale within the selected VPC. The selected VPC is also where the load balancer targets must be hosted unless routing to Lambda or on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view [target groups](#).

vpc-0d1bb5239b4840bd3 (VPC-India)  
192.168.100.0/24

**IP pools** Info

You can optionally choose to configure an IPAM pool as the preferred source for your load balancers IP addresses. Create or view Pools in the [Amazon VPC IP Address Manager console](#).

☐ Use IPAM pool for public IPv4 addresses

The IPAM pool you choose will be the preferred source of public IPv4 addresses. If the pool is depleted IPv4 addresses will be assigned by AWS.

**Availability Zones and subnets** Info

Select at least two Availability Zones and a subnet for each zone. A load balancer node will be placed in each selected zone and will automatically scale in response to traffic. The load balancer routes traffic to targets in the selected Availability Zones only.

☒ ap-south-1a (aps1-az1)

**Subnet**

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.

subnet-05dc525c3d56d0446 (Public-Subnet-1A)  
192.168.100.0/26  
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (-/)

☒ ap-south-1b (aps1-az3)

**Subnet**

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.

subnet-0827e46e96a25e093 (Public-Subnet-1B)  
192.168.100.64/26  
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (-/)

- Attach ALB-SG security group to the load balancer.

EC2 > Load balancers > Create Application Load Balancer

**Security groups** Info

A security group is a set of firewall rules that control the traffic to your load balancer. Listed security groups are in the same VPC as you selected above. Selected security groups must include at least 1 inbound rule and 1 outbound rule to allow traffic to your load balancer.

**Security groups**

Select up to 5 security groups

ALB-SG (sg-05f31770a2c68a9f5) ✕  
Inbound rule (1) Outbound rule (1)

**Listeners and routing** Info

A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

**Listener HTTP:80** Remove

**Protocol** HTTP **Port** 80  
1-65535

**Default action** Info

The default action is used if no other rules apply. Choose the default action for traffic on this listener.

- Configure the listener on **HTTP 80** and forward traffic to **TG-India**.

EC2 > Load balancers > Create Application Load Balancer

**Listeners and routing** Info

A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

**Listener HTTP:80** Remove

**Protocol** HTTP **Port** 80  
1-65535

**Default action** Info

The default action is used if no other rules apply. Choose the default action for traffic on this listener.

**Routing action**

☒ Forward to target groups ☐ Redirect to URL ☐ Return fixed response

**Forward to target group** Info

Choose a target group and specify routing weight or [create target group](#).

**Target group** TG-India HTTP ⌚

Weight 1 0-999 Percent 100%

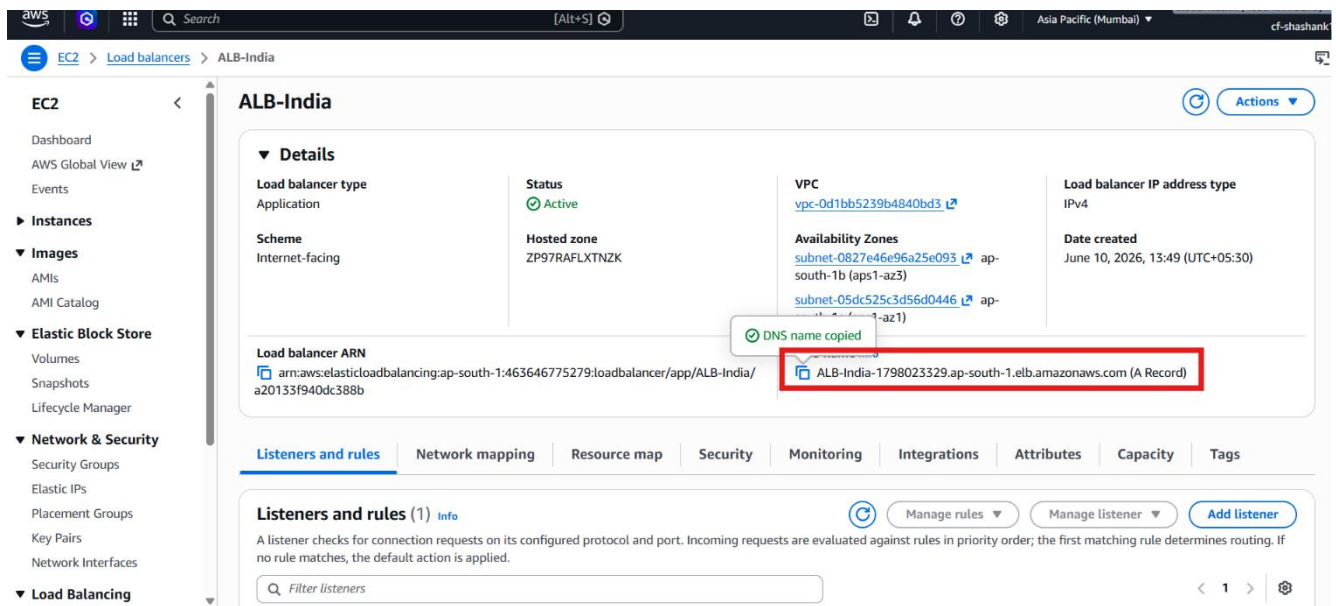
**Target group stickiness** Info

☐ Turn on target group stickiness

Enables the load balancer to bind a user's session to a specific target group. To use stickiness the client must support cookies. If you want to bind a user's session to a specific target, turn on the Target Group attribute Stickiness.

**Listener tags - optional**

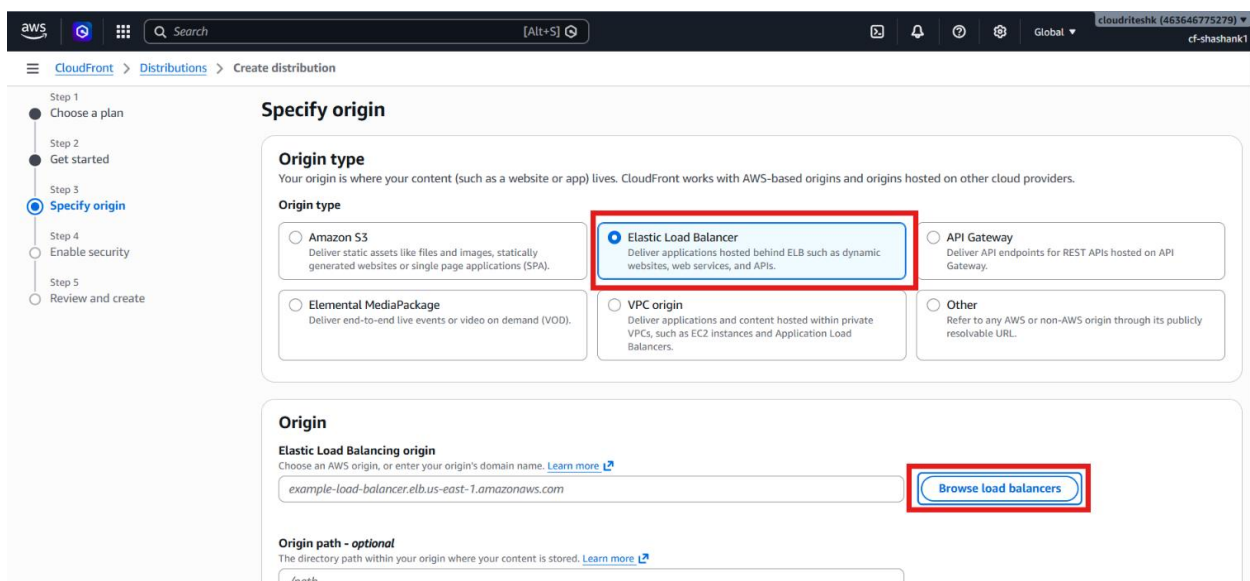
- Create the matching US target group and Application Load Balancer in **VPC-USA**.
- Verify the India ALB DNS name opens the website and alternates between the two India web servers.



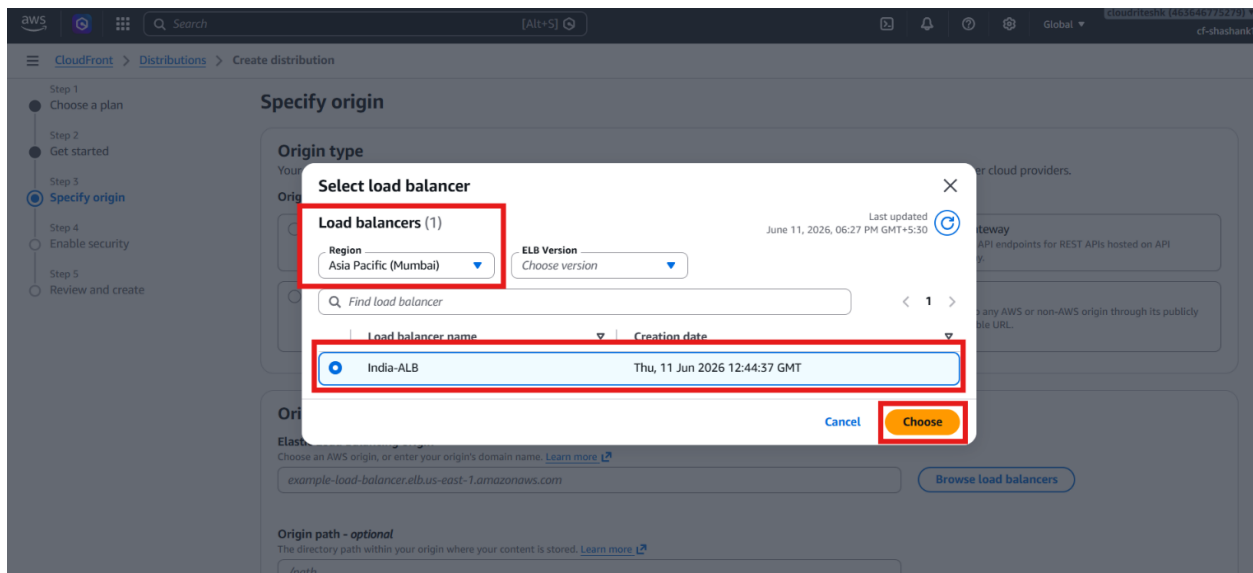
- Verify the US ALB DNS name opens the USA website and alternates between the two US web servers.

### 3. Create the CloudFront Distribution

- Navigate to **Amazon CloudFront**.
- Click **Create Distribution**.



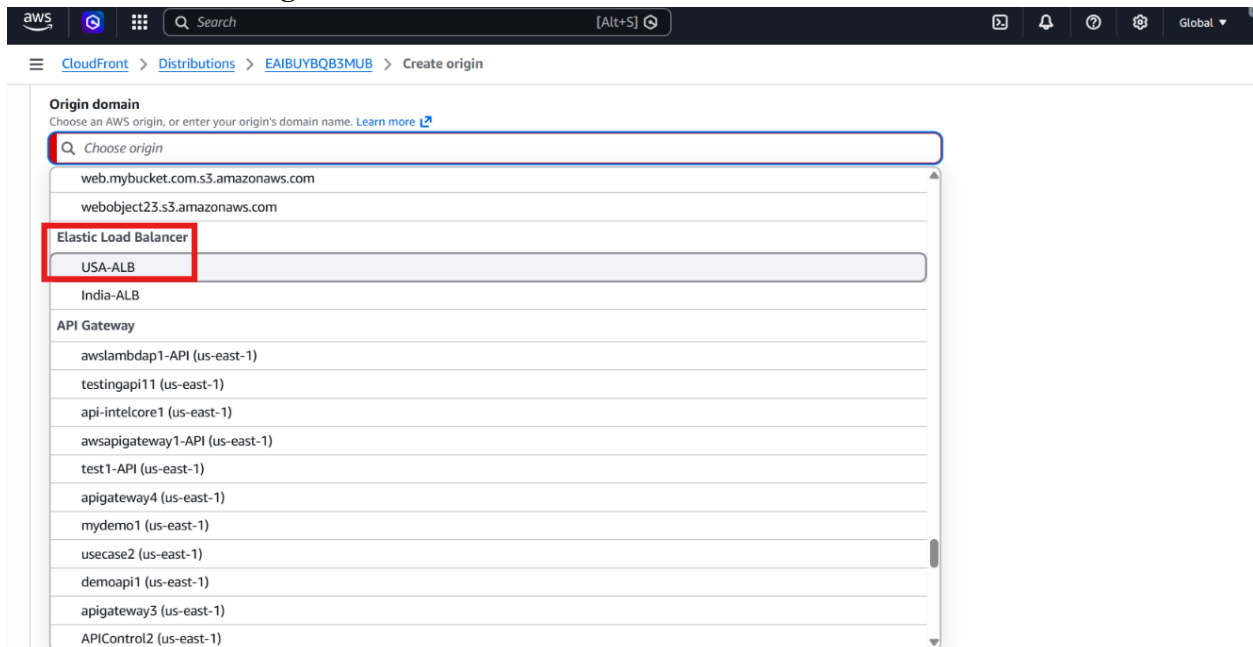
- Select the Mumbai load balancer as the origin.



- Configure **HTTP Only** communication.
- Set cache behavior according to lab requirements.
- Click **Create Distribution**.
- Wait for deployment.

#### 4. Add the Backup Origin

- Open the CloudFront distribution.
- Navigate to **Origins**.
- Click **Create Origin**.



- Select the USA load balancer.
- Click **Create Origin**.

## 5. Create an Origin Group

### Configuration Steps

- Navigate to **Origins**.
- Click **Create Origin Group**.
- Select both load balancers.

**Settings**

**Origins**  
Choose the origins for this group, then put them in priority order.

Choose origins to add to group

1: India-ALB-1649811358.ap-south-1.elb.amazonaws.com-mq9i7xkiqr (primary)

2: usa-alb-96017971.us-east-1.elb.amazonaws.com

**Name**  
Enter a name for this origin group.

India-USA

**Failover criteria**  
Select the origin errors to use as failover criteria.

- ☒ 400 Bad Request
- ☒ 403 Forbidden
- ☒ 404 Not found
- ☒ 416 Range Not Satisfiable
- ☒ 429 Too Many Requests
- ☒ 500 Internal server error
- ☒ 502 Bad gateway
- ☒ 503 Service unavailable
- ☒ 504 Gateway timeout

**Origin selection criteria**  
Choose how the origins in this group are selected.

- Configure the Mumbai load balancer as **Primary Origin**.
- Configure the Virginia load balancer as **Secondary Origin**.
- Select the failover status codes.
- Click **Create Origin Group**.

## 6. Associate the Origin Group with CloudFront Behavior

- Navigate to **Behaviors**.
- Select the default behavior.
- Click **Edit**.

**cloudfreeks-hosting** **Standard** [View metrics](#)

**Details**

Distribution domain name  
d39x0567svij19.cloudfront.net

Billing  
Pay-as-you-go  
[Switch to a plan](#)

ARN  
arn:aws:cloudfront::463646775279:distribution/EAIBUYBQB3MUB

Last modified  
Deploying

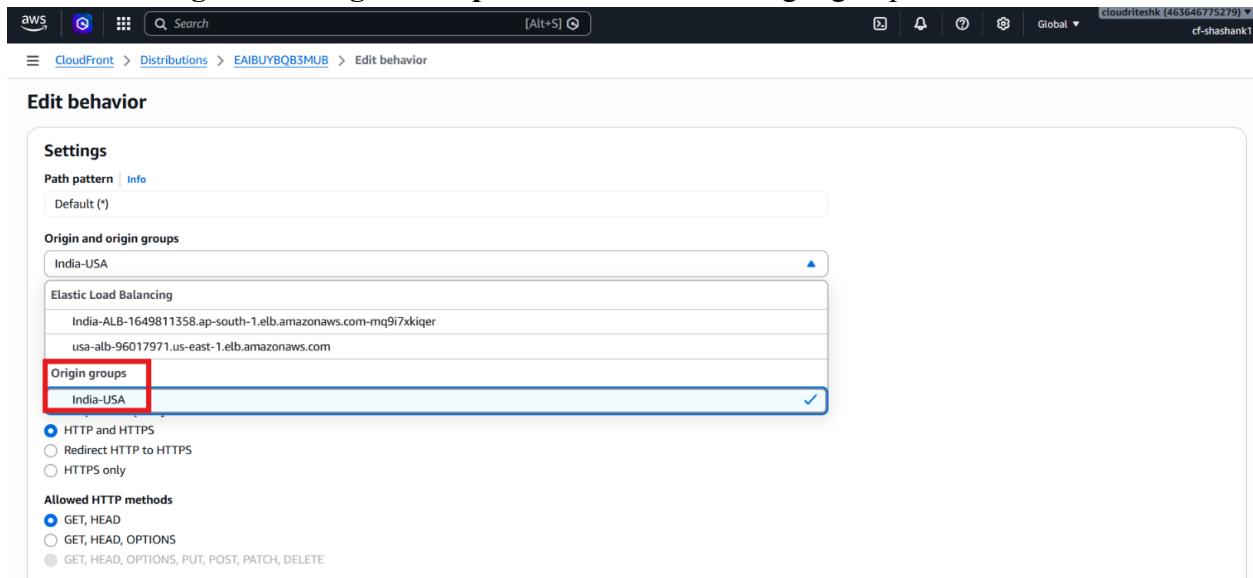
General | Security | Origins | **Behaviors** | Error pages | Invalidation | Logging | Tags

**Behaviors (1/1)** [Save](#) [Move up](#) [Move down](#) [Edit](#) [Delete](#) [Create behavior](#)

Filter behaviors by property or value

Preced...	Path pattern	Origin or origin group	Viewer protocol policy	Cache policy name	Origin request policy na...	Response headers policy...
0	Default (*)	India-ALB-1649811358.a...	Redirect HTTP to HTTPS	UseOriginCacheControlHeader	-	-

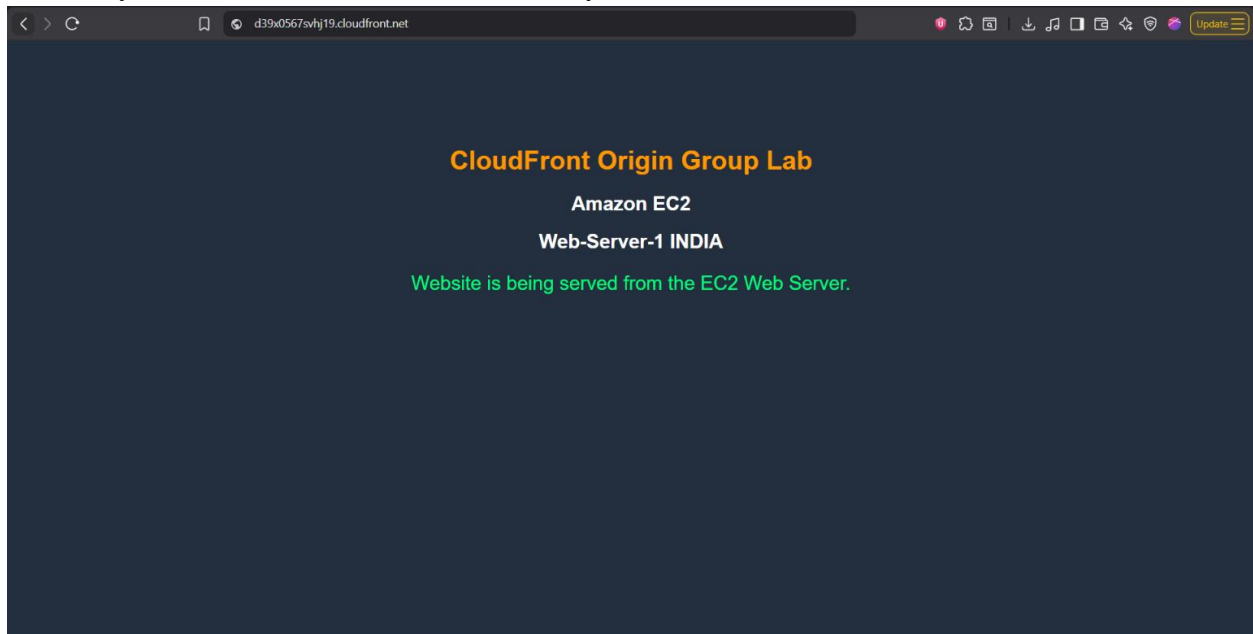
- Under **Origin and Origin Groups**, select the created origin group.



- Change **Cache Policy** to **CacheDisabled** only for lab purpose.
- Click **Save Changes**.

## 7. Verify Normal Traffic Flow

- Open the CloudFront domain name.
- Verify the India website loads successfully.



- Confirm requests are being served through the Mumbai load balancer.

## 8. Test Regional Failover

### Failover Steps

- Navigate to **EC2** → **Load Balancers**.

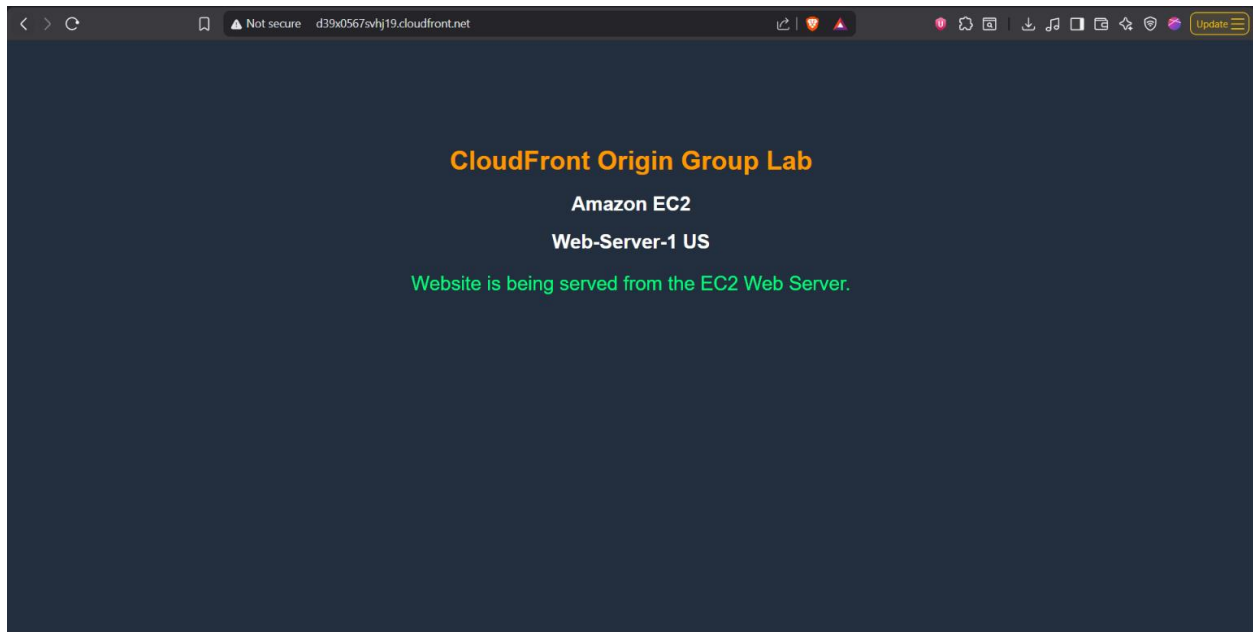
- Open the Mumbai load balancer.
- Locate the attached security group.

The screenshot shows the AWS Management Console interface for EC2 instances. On the left, the navigation pane is visible with 'Instances' selected. The main content area shows a list of instances. The 'Web-Server-1' instance (ID: i-0304afc313c0b8def) is selected. Below the instance list, the 'Security' tab is active, showing the 'Security details' section. Under 'Security groups', the group 'sg-0196fe968d5f848d7 (launch-wizard-1084)' is listed and highlighted with a red box.

- Open Inbound Rules.
- Remove the HTTP port 80 rule.
- Save the changes.

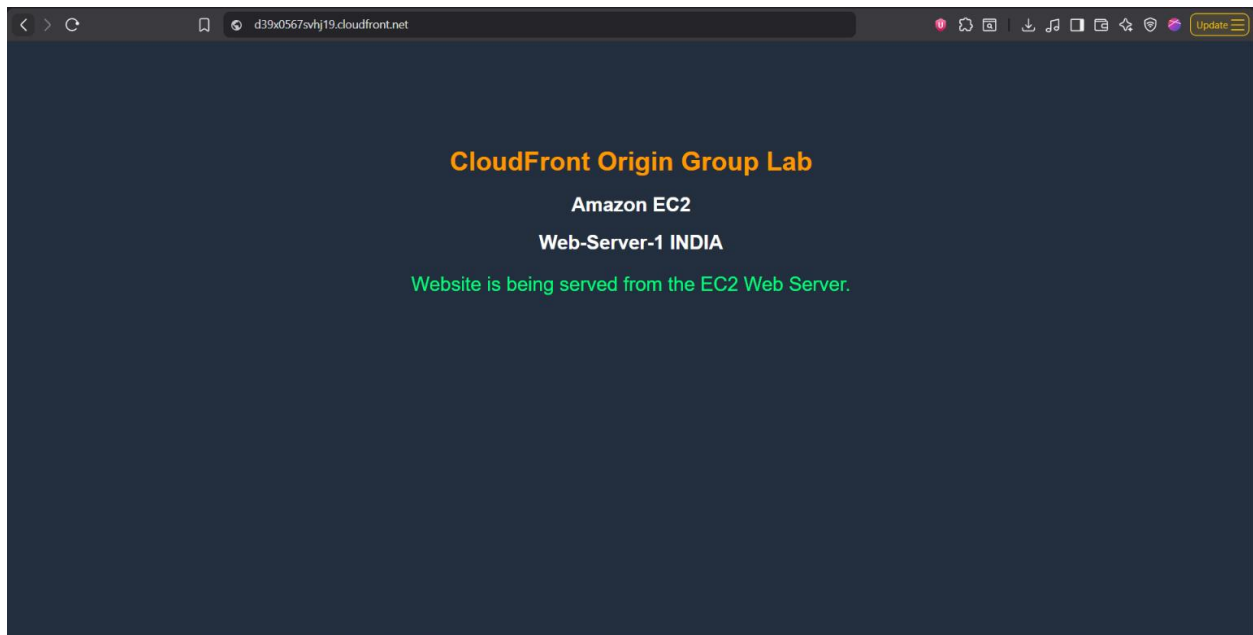
The screenshot shows the 'Edit inbound rules' page for the security group 'sg-0196fe968d5f848d7'. The page displays a list of inbound rules. The first rule is highlighted, showing 'Type' as 'HTTP', 'Protocol' as 'TCP', and 'Port range' as '80'. The 'Delete' button for this rule is highlighted with a red box. At the bottom, there is a warning message: 'Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.'

- Wait for CloudFront health checks.
- Refresh the CloudFront URL.
- Verify the US website appears.



## 9. Restore the Primary Region

- Open the Mumbai load balancer security group.
- Click **Edit Inbound Rules**.
- Re-add HTTP port **80**.
- Save the changes.
- Wait for health checks to succeed.
- Refresh the CloudFront URL.



- Verify traffic returns to the India website.



## Verification

- The Mumbai Application Load Balancer serves as the primary origin.
- The Virginia Application Load Balancer serves as the secondary origin.
- CloudFront Origin Group is configured successfully.
- Traffic automatically fails over during primary origin failure.
- Traffic returns to the primary origin after recovery.
- Multi-region application availability is successfully achieved using **Amazon CloudFront Origin Groups**.